Artefactos BDD · AsesorIA

Historia de usuario en Gherkin, automatización con Cucumber y reflexión

Estudiante	Fecha	Sistema / historia	Herramientas
Manuela Larrea	26 de septiembre de 2026	AsesorIA (proyecto de la Unidad 1) · HU-01 Consulta asistida por IA con protección de datos sensibles	Cucumber-JVM 7.20 · Gherkin (es) · JUnit Platform Suite · Playwright 1.56 · Java 17 · Maven

1. Historia de usuario y criterios de aceptación

La historia proviene del mismo sistema de la Unidad 1, AsesorIA, un asistente conversacional con IA para asesores de servicio de una entidad financiera, inspirado en los asistentes corporativos gobernados que se despliegan en banca. Es la historia central del producto: la que justifica su existencia y la que concentra el riesgo de negocio (respuestas inventadas, fuga de datos sensibles).

Campo	Contenido
ID · Título	HU-01 · Consulta asistida por IA con protección de datos sensibles
Historia	Como asesor de servicio al cliente de la entidad financiera, quiero consultar a AsesorIA la respuesta oficial de un producto en lenguaje natural, para responder al cliente con rapidez y sin exponer datos sensibles.
CA-01	Si la consulta corresponde a una intención del catálogo (10 productos/servicios), el asistente devuelve la respuesta oficial, muestra la intención detectada y una confianza ≥ 0,40, incluso ante paráfrasis.
CA-02	Si la consulta está fuera del dominio, el asistente no inventa una respuesta: indica que no tiene información suficiente y deriva al supervisor (estado fallback).
CA-03	Si la consulta contiene un número de tarjeta válido, el asistente la rechaza, no la procesa con el motor de IA y el historial la registra con el número enmascarado (estado blocked).
Definición de terminado	Los tres criterios tienen al menos un escenario Gherkin automatizado y en verde en el entorno local; el reporte Cucumber (HTML) con capturas queda como evidencia.

2. Especificación Given–When–Then (Gherkin, español)

Los criterios se redactaron primero en lenguaje de negocio y después como escenarios en Gherkin (español), siguiendo las buenas prácticas de especificación: un comportamiento por escenario, pasos declarativos (qué hace el asesor, no qué botón pulsa), datos concretos en los ejemplos, Antecedentes para el contexto compartido y un Esquema del escenario para las paráfrasis. La trazabilidad se materializa con etiquetas: @HU-01 en la característica y @CA-xx en cada escenario, de modo que puede ejecutarse un criterio aislado (mvn test -Dtest=RunCucumberTest -Dcucumber.filter.tags="@CA-03") y el reporte agrupa los resultados por criterio.

```
# language: es
@HU-01 @asesoria
Característica: Consulta asistida por IA con protección de datos sensibles
  Como asesor de servicio al cliente de la entidad financiera
  quiero consultar a AsesorIA la respuesta oficial de un producto en lenguaje natural
  para responder al cliente con rapidez y sin exponer datos sensibles

  Antecedentes:
    Dado que el asesor "asesor.demo" ha iniciado sesión en AsesorIA
    Y se encuentra en la pantalla de chat

  @CA-01
  Escenario: Una consulta del dominio recibe la respuesta oficial con su intención
    Cuando el asesor consulta "¿Cuál es el horario de atención de las sucursales?"
    Entonces el asistente responde con la intención "horario_sucursales"
    Y la respuesta contiene "8:30"
    Y el estado de la respuesta es "ok"
    Y la confianza mostrada es mayor o igual a 0.40

  @CA-01
  Esquema del escenario: Las intenciones del catálogo de productos se reconocen ante paráfrasis
    Cuando el asesor consulta "<consulta>"
    Entonces el asistente responde con la intención "<intencion>"
    Y el estado de la respuesta es "ok"
```

```
  Ejemplos:
    | consulta                                      | intencion         |
    | qué necesito para pedir un préstamo personal  | credito_personal  |
    | perdí mi tarjeta de débito, ¿cómo la bloqueo? | bloqueo_tarjeta   |
    | cuánto puedo retirar del cajero por día       | limites_cajero    |
    | hay un cargo que no reconozco en mi tarjeta   | reporte_fraude    |

@CA-02
Escenario: Una consulta fuera del dominio recibe una respuesta de derivación, no una inventada
  Cuando el asesor consulta "¿Cómo preparo una lasaña?"
  Entonces el asistente indica que no tiene información suficiente
  Y el estado de la respuesta es "fallback"

@CA-03
Escenario: Una consulta con número de tarjeta se bloquea y se registra enmascarada
  Cuando el asesor consulta "El cliente indica que su tarjeta 4111 1111 1111 1111 fue clonada"
  Entonces el asistente rechaza la consulta por contener datos sensibles
  Y el estado de la respuesta es "blocked"
  Y el historial muestra la consulta con el número enmascarado como "**** **** **** 1111"
```

Trazabilidad entre criterios de aceptación, escenarios y pasos verificadores:

Criterio	Escenario(s) Gherkin	Pasos verificadores (glue)	Evidencia observable
CA-01	«Una consulta del dominio recibe la respuesta oficial…» + esquema «Las intenciones del catálogo se reconocen ante paráfrasis» (4 ejemplos)	el asistente responde con la intención {string}; la respuesta contiene {string}; el estado de la respuesta es "ok"; la confianza mostrada es mayor o igual a {double}	Badge de intención, texto de respuesta, badge de confianza en la UI
CA-02	«Una consulta fuera del dominio recibe una respuesta de derivación, no una inventada»	el asistente indica que no tiene información suficiente; el estado de la respuesta es "fallback"	Mensaje de derivación e intención fuera_de_dominio
CA-03	«Una consulta con número de tarjeta se bloquea y se registra enmascarada»	el asistente rechaza la consulta por contener datos sensibles; el estado es "blocked"; el historial muestra la consulta con el número enmascarado como {string}	Respuesta bloqueada sin confianza y fila del historial con «**** **** **** 1111»

3. Automatización con Cucumber e integración en el entorno local

Los escenarios se automatizaron en el mismo proyecto Maven de la Unidad 1 (tests-e2e), de forma que conviven con la suite JUnit/Playwright y comparten los Page Objects. Cucumber se ejecuta sobre JUnit Platform, por lo que Surefire, los reportes XML y los parámetros -D existentes aplican sin cambios.

Componente	Archivo	Responsabilidad
Especificación	src/test/resources/features/consulta_asistente.feature	Historia, antecedentes y 4 escenarios (7 ejecuciones) en Gherkin español, con etiquetas de trazabilidad.
Glue (step definitions)	bdd/ConsultaAsistenteSteps.java	Traduce cada paso a acciones sobre los Page Objects (LoginPage, ChatPage, HistorialPage) y aserciones con auto-espera; hooks @Before/@After crean un contexto de navegador por escenario, adjuntan la captura al reporte y graban video.
Runner	bdd/RunCucumberTest.java	@Suite + @IncludeEngines("cucumber"): descubre las features, fija el paquete glue, el filtro de etiquetas y los plugins (pretty, html, json).
Dependencias	pom.xml	cucumber-java, cucumber-junit-platform-engine 7.20.1 y junit-platform-suite (versión gestionada por el BOM de JUnit 5.11).

```
@Cuando("el asesor consulta {string}")
public void elAsesorConsulta(String consulta) { ultimaRespuesta = chat.send(consulta); }

@Entonces("el asistente responde con la intención {string}")
public void elAsistenteRespondeConLaIntencion(String intencion) {
  assertThat(chat.intentOf(ultimaRespuesta)).hasText(intencion);        // aserción con auto-espera
}
```

- **Levantar el SUT:** cd sut && source .venv/bin/activate && uvicorn app.main:app --port 8000.
- **Ejecutar los escenarios:** cd tests-e2e && mvn test -Dtest=RunCucumberTest (o mvn test para toda la suite: 32 pruebas de la Unidad 1 + 7 escenarios BDD). Con -Dheadless=false -DslowMo=300 se ve el navegador.
- **Resultado esperado:** 7 scenarios (7 passed), 38 steps (38 passed) en la consola; reporte navegable en target/cucumber/reporte.html con una captura por escenario, JSON en target/cucumber/reporte.json (integrable en CI) y videos en target/evidence/videos-bdd.

4. Reflexión: BDD como herramienta de comunicación y validación colaborativa

Escribir la historia en Given–When–Then obligó a decidir, antes de tocar código, qué significa «funciona» para el negocio: no bastaba con que el asistente respondiera, sino que respondiera lo oficial, reconociera que no sabe y protegiera los datos del cliente. Esos tres verbos se convirtieron en los criterios CA-01 a CA-03 y, sin traducción intermedia, en escenarios ejecutables. Ahí está el primer valor de BDD: el mismo texto sirve a la persona de producto para validar el alcance, al desarrollador para saber qué construir y al tester para saber qué automatizar; desaparece la brecha entre el criterio de aceptación escrito en una tarjeta y el caso de prueba escrito en otra herramienta.

El segundo valor es la conversación que provoca. Al redactar CA-02 surgió la pregunta de qué debe hacer el asistente cuando no está seguro, y la respuesta (derivar, no inventar) es una decisión de negocio que en un proyecto de IA suele quedar implícita en un umbral técnico. En un contexto bancario regulado, escenarios como CA-03 documentan de forma legible para auditoría cómo se protege la información del cliente, algo que un test unitario no comunica. En la Unidad 1 el defecto del umbral se encontró diseñando pruebas negativas; en BDD ese caso se habría discutido con negocio en la sesión de refinamiento, antes de escribir una línea.

También aparecen sus límites: Gherkin no reemplaza a la pirámide de pruebas (los escenarios son lentos y de alto nivel; la precisión del clasificador se sigue midiendo con pruebas unitarias y datasets), exige disciplina para no degradar en scripts de interfaz («hago clic en…») y solo aporta valor si las tres perspectivas participan realmente en su redacción. Con esas condiciones, BDD alinea negocio, pruebas automatizadas y calidad del producto porque convierte los criterios de aceptación en una especificación viva que se ejecuta en cada cambio y documenta el comportamiento vigente del sistema.